Guião · Miguel

Motor do mercado (livro de ordens, colares, leilão) e integração

SAD · Grupo 2 · Volatility Interruptions and Crash Contagion in Euronext Lisbon · entrega a 2 de dezembro. Os nomes vêm de `docs/CONTRATO.md`.

Resumo das tuas semanas

Semana	Entrega concreta
1 (17–24 out)	Livro de ordens em Java (Order, Stock, matching) **e os testes do livro a passar**, antes de qualquer agente. Depois os noise traders. **Até quinta o preço mexe num gráfico.**
2 (24–31 out)	Statechart do Stock com colares (±3%/±8%), reservas de 3/10 min e leilão de reabertura, com testes. Integrar o fundamentalista do João e a instrumentação da Margarida.
3 (31 out–7 nov)	Integrar tudo (chartists, criador de mercado, notícias, dashboard). **Fase 1 a correr.**
4 (7–14 nov)	Velocidade do modelo e apoio à calibração.
5 (14–21 nov)	Corrigir o que a verificação da Matilde encontrar; apoio às corridas finais.
6 (21–28 nov)	Congelar o modelo; introdução, problema e livro de ordens no relatório.

0. Como trabalhar (lê isto primeiro)

Uma vez só

1. Aceita o convite do GitHub que chega ao teu email (repositório **sad-euronext-g2**).
2. Instala o **GitHub Desktop** (desktop.github.com) e entra com a tua conta.
3. No GitHub Desktop: **File → Clone repository → sad-euronext-g2 → Clone**. Fica uma pasta no teu PC (por defeito em Documentos\GitHub\sad-euronext-g2).
4. Confirma com o grupo que **todos usam a mesma versão do AnyLogic** (Help → About). Um modelo gravado numa versão mais recente pode não abrir numa mais antiga.

Todos os dias

1. GitHub Desktop: **Fetch origin → Pull** (trazes o que os outros fizeram).
2. AnyLogic: **File → Open** e abres o teu `.alp` dentro da pasta do repositório. Não reinstalas nada: o GitHub só mantém a pasta sincronizada.
3. Trabalhas e gravas (Ctrl+S).
4. GitHub Desktop: escreves uma mensagem curta (ex.: `João: fundamentalista com statechart`) → **Commit to main → Push origin**.

Onde trabalhas

- `model/Euronext_Main.alp` é o modelo principal e **só o Miguel o edita**.
- Tu trabalhas em `sandbox/miguel/`. Depois do Miguel publicar o modelo base, copia `model/Euronext_Main.alp` para a tua pasta com outro nome (ex.: `sandbox/miguel/miguel.alp`) e abre esse. Na tua cópia podes mexer em tudo, incluindo no Main.
- Quando a tua peça funcionar: commit + push e avisa no grupo. O Miguel abre os dois modelos no AnyLogic e, no painel **Projects**, copia o teu agente (Ctrl+C) e cola-o no modelo principal (Ctrl+V). Isto só funciona sem erros se usares os nomes do `docs/CONTRATO.md`.
- Se estragares a tua cópia: GitHub Desktop → separador **History** → botão direito num commit antigo → **Revert**. Ou voltas a copiar o modelo principal.

AnyLogic em 1 minuto

Coisa	Para que serve	Onde está
Agent type	um tipo de participante (ex.: `Fundamentalist`)	Palette → Agent → arrastar **Agent**
Population	muitos agentes do mesmo tipo dentro do Main	assistente do Agent → *Population of agents*
Parameter	valor fixo que muda de cenário para cenário (ex.: `nChartists`)	Palette → Agent → Parameter
Variable	valor que muda durante a simulação (ex.: `cash`)	Palette → Agent → Variable
Function	bloco de código que se chama pelo nome	Palette → Agent → Function
Event	código que corre num instante ou de X em X minutos	Palette → Agent → Event
Statechart	os estados de um agente e o que o faz mudar de estado	Palette → Statechart
Process Modeling Library	blocos de filas (Enter, Queue, Hold, Delay, Sink…)	Palette → Process Modeling Library
`main`	dentro de qualquer agente, `main` dá acesso ao Main (ex.: `main.stock.mid()`)	—

Como desenhar um statechart

1. Abre o agente: duplo clique no tipo (ex.: `Fundamentalist`) no painel **Projects**.
2. Palette → **Statechart** → arrasta **Statechart Entry Point**. O nome fica `statechart`; não o mudes (o Miguel chama `statechart` em todos os agentes).
3. Arrasta um **State**, encosta-lhe a ponta do Entry Point e dá-lhe o nome em **Name**.
4. Transição: na Palette faz **duplo clique em Transition** (fica em modo de desenho), clica na borda do estado de origem e depois na borda do estado de destino.
5. Clica na transição e escolhe **Triggered by**: **Rate** (acontece em média X vezes por minuto, ao acaso: "de vez em quando decide"); **Timeout** (X minutos depois de entrar no estado; com Timeout 0 o agente faz a ação e volta logo); **Message** (quando chega uma mensagem: Message type `String`, Fire transition *On particular message*, Message `"HALT"`); **Condition** (usada à saída de um Branch).
6. **Branch** (o losango na Palette Statechart) serve para decidir. Cada seta que sai dele tem uma Condition; uma tem de ser **Default** (a que se segue se nenhuma outra for verdadeira).
7. Código: cada estado tem **Entry action** (corre ao entrar) e cada transição tem **Action** (corre ao passar) e **Guard** (condição extra para a transição poder acontecer).
8. **Estado composto**: desenha um State grande e arrasta outros para dentro; dentro dele põe um **Initial State Pointer** apontado para o primeiro estado. Uma seta que sai do estado grande vale para todos os de dentro: é assim que se faz o `HALT` (de qualquer estado, o agente vai para `Halted`).
9. Testar: carrega em Run; na barra de cima da janela da simulação escolhe o agente (ex.: `fundamentalists[0]`) e vês o estado ativo a verde. Para escrever na consola: `traceln("texto " + x);`.

Este guião tem o código para colar. Onde aparece **Entry action**, **Action** ou o nome de uma função, copia o bloco cinzento para esse campo. Se o AnyLogic não aceitar `statechart.receiveMessage(msg)`, usa `statechart.fireEvent(msg)`, que faz o mesmo.

Os professores avisaram que o livro de ordens vai ser **sobretudo código Java** e que deve ser **construído e testado primeiro**. Por isso cada ordem é um objeto Java (não um agente) e o livro são duas filas com prioridade (`PriorityQueue`) dentro do `Stock`. Isto tem ainda outra vantagem: um modelo cria milhões de ordens, e se cada uma fosse um agente era provável ultrapassar os limites da versão PLE do AnyLogic.

Continua a ser teoria das filas: cada lado do livro é uma fila servida por prioridade de preço e, no mesmo preço, por ordem de chegada (FCFS); as ordens expiram (desistência) e, durante a reserva, ficam retidas numa fila de espera até ao leilão.

1. Criar o modelo (semana 1)

1. AnyLogic: **File** → **New** → **Model**. Model name `Euronext_Main`; Location: a pasta `model/` do repositório; **Model time units: minutes**.
2. No `Main` cria os **parâmetros** (Palette → Agent → Parameter): `nNoise` int 30, `initialPrice` 0.5, `tickSize` 0.0001, `haltShort` 3, `haltLong` 10, `noiseRate` 1, `noiseSigma` 0.002, `orderLife` 35 (minutos de vida média de uma ordem), `maWindow` 50 (memória da média móvel, em minutos), `runTests` boolean false.
3. E as **variáveis** (Palette → Agent → Variable): `fundamentalValue` double (Initial value `initialPrice`), `nextOrderId` long 0, `dynamicThreshold` double 0.03, `staticThreshold` double 0.08, `collarsActive` boolean true.
4. O resto do Main (cenários, ficheiros de saída e KPIs: `collarSet`, `label`, `runId`, `logEvent`…) é feito pela **Margarida** na semana 2, na cópia dela. Tu só o integras (secção 6).
5. Main → **On startup**: `if (runTests) stock.testBook();`

Funções do Main

roundTick (Returns value, double; argumento `p` double):

```
return Math.max(tickSize, Math.round(p / tickSize) * tickSize);
```

onFill (Just action; argumentos `t` Agent, `side` int, `q` int, `p` double). Atualiza o dinheiro e as ações de quem negociou. Descomenta uma linha por cada tipo de agente que entrar no modelo:

```
if (t instanceof NoiseTrader) { NoiseTrader x = (NoiseTrader) t; x.position += side * q; x.cash -= side * q * p; }
// else if (t instanceof Fundamentalist) { Fundamentalist x = (Fundamentalist) t; x.position += side * q; x.cash -= si
// else if (t instanceof Chartist) { Chartist x = (Chartist) t; x.position += side * q; x.cash -= side * q * p; }
// else if (t instanceof MarketMaker) { MarketMaker x = (MarketMaker) t; x.position += side * q; x.cash -= side * q *
```

broadcast (Just action; argumento `msg` String). Entrega a mensagem a todos; descomenta à medida que entram:

```
// for (Fundamentalist a : fundamentalists) a.statechart.receiveMessage(msg);
// for (Chartist a : chartists) a.statechart.receiveMessage(msg);
// marketMaker.statechart.receiveMessage(msg);
// news.statechart.receiveMessage(msg);
traceln(time() + "  " + msg);
```

2. Classe Order (Java)

Projects → botão direito em `Euronext_Main` → **New** → **Java Class** → Name `Order` → Finish. Apaga o que o AnyLogic escreveu entre as chavetas da classe e cola os campos:

```
public class Order implements java.io.Serializable {
    public long id;                          // cresce com o tempo: serve de desempate (quem chegou primeiro)
    public int side;                         // 1 = compra, -1 = venda
    public double price;                     // preço-limite (€)
    public int qty;                          // quantidade por executar
    public double tSubmit;                   // instante de submissão
    public double expiry;                    // se não for executada até aqui, é cancelada (desistência)
    public com.anylogic.engine.Agent trader;   // quem enviou (null nos testes)
}
```

3. Agente Stock: o livro de ordens

1. Palette → Agent → arrasta **Agent** → **Single agent** → agent type `Stock`, agent name `stock` → Finish.
2. Dentro de `Stock`, variáveis: `lastPrice`, `refPrice` e `ma` (double, Initial value `main.initialPrice`), `halted` boolean false, `volume` int 0, `lastStaticDir` int 0, `haltStart` double, `prePrice` double.
3. Stock → Properties → **Advanced Java** → **Additional class code**: cola as duas filas do livro e a fila de espera da reserva:

```
// Compras: preço mais alto primeiro; no mesmo preço, quem chegou primeiro (id menor)
java.util.PriorityQueue<Order> bids = new java.util.PriorityQueue<>(
    (a, b) -> a.price != b.price ? Double.compare(b.price, a.price) : Long.compare(a.id, b.id));
// Vendas: preço mais baixo primeiro; no mesmo preço, quem chegou primeiro
java.util.PriorityQueue<Order> asks = new java.util.PriorityQueue<>(
    (a, b) -> a.price != b.price ? Double.compare(a.price, b.price) : Long.compare(a.id, b.id));
// Ordens que chegam durante uma reserva: esperam pelo leilão de reabertura
java.util.ArrayList<Order> pending = new java.util.ArrayList<>();
```

Funções do Stock

order (Just action; argumentos `trader` Agent, `side` int, `price` double, `qty` int). É esta que os investidores chamam:

```
Order o = new Order();
o.id = main.nextOrderId++;
o.side = side;
o.price = main.roundTick(price);
o.qty = qty;
o.tSubmit = time();
o.expiry = time() + main.orderLife * uniform(0.3, 1.7);   // desistência (pressuposto)
o.trader = trader;
if (halted) pending.add(o);            // durante a reserva fica retida até ao leilão
else { addToBook(o); match(); }
```

addToBook (Just action; argumento `o` Order): `if (o.side == 1) bids.add(o); else asks.add(o);`

bestBid, **bestAsk**, **mid** e **spread** (todas Returns value, double):

```
// bestBid
return bids.isEmpty() ? Double.NaN : bids.peek().price;
// bestAsk
return asks.isEmpty() ? Double.NaN : asks.peek().price;
// mid
double b = bestBid(), a = bestAsk();
if (Double.isNaN(b) || Double.isNaN(a)) return lastPrice;
return (a + b) / 2;
// spread
return bestAsk() - bestBid();
```

cancelAll (Just action; argumento `a` Agent):

```
bids.removeIf(o -> o.trader == a);
asks.removeIf(o -> o.trader == a);
```

execute (Just action; argumentos `b` Order, `a` Order, `p` double, `q` int). `b` e `a` são sempre a cabeça de cada fila:

```
lastPrice = p;
volume += q;
b.qty -= q;
a.qty -= q;
main.onFill(b.trader, 1, q, p);
main.onFill(a.trader, -1, q, p);
if (b.qty == 0) bids.poll();
if (a.qty == 0) asks.poll();
```

collarHit (Returns value, boolean; argumento `p` double). Na semana 1 o corpo é só `return false;`. O código verdadeiro está na secção 5.

match (Just action):

```
while (!halted && !bids.isEmpty() && !asks.isEmpty()) {
    Order b = bids.peek();
    Order a = asks.peek();
    if (b.price < a.price) break;                 // não há cruzamento
    double p = (a.id < b.id) ? a.price : b.price; // preço de quem já estava no livro
    if (collarHit(p)) return;                      // abre reserva; a ordem espera pelo leilão
    execute(b, a, p, Math.min(a.qty, b.qty));
}
```

Eventos do Stock

- **matchEvent**: Trigger type *Timeout*, Mode **User control**, Action `match();`
- **purgeEvent** (desistências): Mode *Cyclic*, First occurrence 1, Recurrence 1, Action `bids.removeIf(o -> o.expiry < time()); asks.removeIf(o -> o.expiry < time());`
- **dayEvent** (abertura de cada dia): Mode *Cyclic*, First occurrence 510, Recurrence 510, Action `refPrice = lastPrice; lastStaticDir = 0;`
- **maEvent** (média móvel de ~50 min para os chartists): Mode *Cyclic*, First occurrence 1, Recurrence 1, Action `ma = ma + (lastPrice - ma) / main.maWindow;`

4. Testar o livro ANTES dos agentes (semana 1)

É o que os professores pediram. Os testes metem ordens conhecidas e confirmam o resultado. Correm com `runTests = true` e `nNoise = 0` (sem agentes a baralhar). Na consola deve aparecer só **OK**.

check (Just action; argumentos `name` String, `ok` boolean): `traceln((ok ? "OK " : "FALHOU ") + name);`

testBook (Just action):

```
bids.clear(); asks.clear(); pending.clear();
lastPrice = 0.50; refPrice = 0.50; volume = 0;
boolean collars = main.collarsActive;
main.collarsActive = false;

// 1. Sem cruzamento: nada é executado e o spread é a diferença
order(null, 1, 0.50, 100);
order(null, -1, 0.51, 100);
check("1. sem cruzamento: volume 0", volume == 0);
check("1. spread 0.01", Math.abs(spread() - 0.01) < 1e-9);

// 2. Execução parcial ao preço de quem já estava no livro
order(null, -1, 0.50, 40);
check("2. executa 40 a 0.50", volume == 40 && Math.abs(lastPrice - 0.50) < 1e-9);
check("2. sobram 60 na compra", bids.peek().qty == 60);

// 3. Prioridade de tempo no mesmo preço
order(null, 1, 0.50, 10);              // chega depois, mesmo preço
order(null, -1, 0.50, 65);             // leva os 60 da 1.a compra e 5 da 2.a
check("3. FCFS: volume 105", volume == 105);
check("3. FCFS: a 2.a compra fica com 5", bids.size() == 1 && bids.peek().qty == 5);

// 4. Uma compra grande varre dois níveis de venda
order(null, -1, 0.52, 100);
order(null, 1, 0.52, 150);             // 100 a 0.51 e 50 a 0.52
check("4. varre níveis: volume 255", volume == 255);
check("4. último preço 0.52", Math.abs(lastPrice - 0.52) < 1e-9);
check("4. sobram 50 a 0.52", Math.abs(asks.peek().price - 0.52) < 1e-9 && asks.peek().qty == 50);

// 5. Colar dinâmico: um negócio a -3.8% não é executado
main.collarsActive = true;
order(null, -1, 0.40, 100);            // cruza com a compra de 5 a 0.50: 0.50/0.52 - 1 = -3.8%
check("5. colar: nada executado", volume == 255);

// 6. Cancelamento
cancelAll(null);
check("6. cancelAll limpa o livro", bids.isEmpty() && asks.isEmpty());

main.collarsActive = collars;
```

Se um teste FALHAR, não avances para os agentes: corrige primeiro. O teste 5 só passa depois de fazeres a secção 5 (até lá falha, e é normal). Sempre que mexeres no livro, volta a correr os testes.

5. Noise traders e gráfico (semana 1)

1. Palette → Agent → **Agent** → **Population of agents** → type `NoiseTrader`, population `noiseTraders`, Initial number `nNoise`.
2. Variáveis do NoiseTrader: `cash` double 1e9, `position` int 0.
3. Event `trade`: Trigger type **Rate**, Rate `main.noiseRate` (per minute). Action:

```
int side = randomTrue(0.5) ? 1 : -1;
double p = main.stock.mid() * (1 + normal(main.noiseSigma, 0));   // AnyLogic: normal(sigma, média)
main.stock.order(this, side, p, uniform_discr(100, 1000));
```

1. Palette **Analysis** → **Time Plot** no Main: Value `stock.lastPrice` (título Preço) e `fundamentalValue`; Time window 510.
2. Projects → **Simulation** → Model time: Stop **At specified time** 61200 (120 dias). **Run** com `runTests` false e `nNoise` 30.
3. Confirma que o preço mexe. Commit `Miguel: livro de ordens testado, o preço mexe` e **avisa o grupo**: toda a gente começa a trabalhar a partir deste ficheiro.

6. Colares, reservas e leilão (semana 2)

Regras da proposta: reserva se o preço sair de ±3% do último preço (**dinâmico**) ou de ±8% do preço de referência (**estático**; abertura ou último leilão). A reserva dura 3 min, ou 10 min se o estático voltar a disparar no mesmo sentido. Depois há um leilão de reabertura.

Substitui o corpo de **collarHit**:

```
if (!main.collarsActive) return false;
boolean dyn  = Math.abs(p / lastPrice - 1) > main.dynamicThreshold;
boolean stat = Math.abs(p / refPrice - 1) > main.staticThreshold;
if (!dyn && !stat) return false;
int dir = p > lastPrice ? 1 : -1;
if (stat && dir == lastStaticDir) statechart.receiveMessage("STATIC_REPEAT");
else if (stat) { lastStaticDir = dir; statechart.receiveMessage("STATIC"); }
else statechart.receiveMessage("DYNAMIC");
return true;
```

Statechart do Stock

```
             +--"DYNAMIC"------> ReservDynamic --Timeout haltShort--+
             |                                                      v
 Continuous -+--"STATIC"-------> ReservStatic  --Timeout haltShort--> Auction --Timeout 0.01--> Continuous
             |                                                      ^          (runAuction(); endHalt();)
             +--"STATIC_REPEAT"> ReservStatic10 -Timeout haltLong---+
```

Estado	Entry action
Continuous (inicial)	`halted = false; matchEvent.restart(0);`
ReservDynamic, ReservStatic, ReservStatic10	`startHalt();`
Auction	`for (Order o : pending) addToBook(o); pending.clear();` (as ordens retidas entram no livro)

As 3 transições que saem de Continuous: Triggered by **Message**, String, *On particular message* com `"DYNAMIC"`, `"STATIC"` e `"STATIC_REPEAT"`. Timeouts: `main.haltShort`, `main.haltShort` e `main.haltLong`. Auction → Continuous: Timeout `0.01`, Action `runAuction(); endHalt();`.

startHalt e **endHalt** (Just action):

```
// startHalt
halted = true;                // a partir daqui as ordens novas vão para 'pending'
haltStart = time();
prePrice = lastPrice;
main.nHalts++;
main.broadcast("HALT");

// endHalt
main.haltsFile.println(main.label + ";" + main.runId + ";" + haltStart + ";" + time() + ";"
                       + prePrice + ";" + lastPrice);
main.timeHalted += time() - haltStart;
main.broadcast("REOPEN");
```

As linhas com `main.nHalts`, `main.haltsFile`, `main.label`, `main.runId` e `main.timeHalted` usam a instrumentação da Margarida. Até a integrares, deixa-as comentadas com `//`.

runAuction (Just action). Escolhe o preço que maximiza a quantidade negociada e executa tudo a esse preço único:

```
double bestP = lastPrice;
int bestQ = 0;
java.util.TreeSet<Double> cands = new java.util.TreeSet<>();
for (Order o : bids) cands.add(o.price);
for (Order o : asks) cands.add(o.price);
for (double p : cands) {
    int dem = 0, sup = 0;
    for (Order o : bids) if (o.price >= p) dem += o.qty;
    for (Order o : asks) if (o.price <= p) sup += o.qty;
    int q = Math.min(dem, sup);
    if (q > bestQ || (q == bestQ && q > 0 && Math.abs(p - lastPrice) < Math.abs(bestP - lastPrice))) {
        bestQ = q; bestP = p;
    }
}
while (bestQ > 0 && !bids.isEmpty() && !asks.isEmpty()
       && bids.peek().price >= bestP && asks.peek().price <= bestP) {
    Order b = bids.peek(), a = asks.peek();
    execute(b, a, bestP, Math.min(a.qty, b.qty));
}
refPrice = lastPrice;         // o leilão passa a ser a nova referência estática
```

Testes: volta a correr o `testBook` (agora o teste 5 tem de dar OK). Depois, com agentes, põe `noiseSigma = 0.01` para forçar movimentos grandes: devem aparecer `HALT`/`REOPEN` na consola e o preço fica parado durante 3 min. Volta a pôr 0.002.

7. Integração (semanas 2–3)

1. Quando alguém avisar: Pull e abre o modelo dessa pessoa e o `model/Euronext_Main.alp` (ambos ficam no Projects).
2. No modelo da pessoa, seleciona o tipo de agente (ex.: `Fundamentalist`) → Ctrl+C → seleciona `Euronext_Main` → Ctrl+V.
3. Arrasta o tipo, do Projects para o canvas do Main, para criar a população. Usa o nome do CONTRATO (ex.: `fundamentalists`) e em Initial number o parâmetro (`nFundamentalists`).
4. Copia também os parâmetros novos que essa pessoa criou no Main dela (seleciona-os no canvas → Ctrl+C → Ctrl+V).
5. Descomenta as linhas desse tipo em `onFill` e `broadcast`.
6. **Instrumentação da Margarida** (semana 2): copia do Main dela os parâmetros e variáveis novos, os 2 Text File, o `logEvent` e o texto do *On startup* e do *On destroy*. Depois descomenta as linhas de `startHalt` e `endHalt`.
7. Build (F7) → corrige os erros que aparecerem no painel Problems → corre o `testBook` → Run → Commit `Miguel: integrado X`.

8. Velocidade (semana 4)

- Nas experiências (Parameters Variation) não há animação, por isso correm muito mais depressa.
- Se a Simulation estiver lenta: Run → no fundo da janela, **Virtual time mode** (o botão da seta dupla).
- Se o livro ficar com milhares de ordens, reduz o `orderLife` e diz ao João e à Margarida (é um pressuposto que entra na calibração).
- Confirma os limites da tua licença (PLE) no site da AnyLogic: duração das corridas, número de agentes e que experiências estão disponíveis. Se algum limite apertar, avisa a Margarida.

9. Semanas 5–6

- Semana 5: a Matilde faz os testes de verificação (valores extremos). Corriges o que ela encontrar.

- Último commit com a mensagem `VERSÃO FINAL`. A partir daí ninguém mexe no `model/`.
- No relatório: Introdução, Definição do problema (partem da proposta) e o livro de ordens (secções 3, 4 e 6 deste guião, incluindo a tabela dos testes).